

Práctica 3: Hormiga de Langton Evolucionada

1. Objetivo

El objetivo de esta práctica es trabajar con los mecanismos de herencia, polimorfismo dinámico, plantillas, sobrecarga de operadores y control de excepciones en lenguaje C++.

2. Entrega

La práctica se realizará en dos sesiones de laboratorio:

- Sesión tutorada: del 2 al 5 de marzo.
- Sesión de entrega: del 9 al 12 de marzo.

3. Enunciado

En esta práctica se continúa el desarrollo de código de las prácticas anteriores [1,2]. Se introducen nuevas formas de manejar las condiciones de frontera en la cinta para emular un tamaño infinito y se define una estructura de datos auxiliar para facilitar su implementación.

Las *condiciones de frontera*, o de contorno, son restricciones que definen el comportamiento en los límites de un sistema. Según la forma de definir las restricciones se establecen distintas condiciones de frontera, que darán lugar a comportamientos diferentes durante la simulación. Algunas de estas condiciones de frontera son:

- Frontera periódica. Se considera que los extremos de la cinta son adyacentes en cada dimensión. Por tanto, cuando la hormiga traspasa el borde izquierdo de la cinta reaparece por el borde derecho; cuando traspasa el borde superior reaparece por el borde inferior de la cinta; y así respectivamente en los otros bordes.
- Frontera reflectora. En este caso, cuando la hormiga alcanza una celda en el borde de la cinta y queda orientada hacia el exterior, se modifica su orientación para que en el siguiente paso avance hacia el interior de la cinta.
- Frontera deslizante. Se basa en utilizar una implementación de la cinta que le permita crecer dinámicamente cuando la hormiga traspasa alguno de sus bordes. Así, si la hormiga traspasa el borde izquierdo de la cinta, se incrementa el tamaño de la cinta añadiendo una columna de celdas por el principio; si la hormiga traspasa el borde superior, a la cinta se añade una fila de celdas al principio; y así respectivamente en los otros bordes.

Cuando en un paso de la simulación dos o más hormigas alcanzan la misma celda se produce una situación conocida como *condición de carrera* [4]. En esta situación, en un mismo paso de la simulación la celda va a cambiar de color tantas veces como hormigas hayan coincidido en dicha celda. Además, la secuencia de movimientos de dichas hormigas dependerá del orden en que se actualicen. En esta implementación no se evitan la condición de carrera, pero se utiliza la coincidencia de dos o más hormigas en una misma celda para añadir interacción entre los distintos tipos de hormigas.

4. Notas de implementación

- `Tape`, la clase que representa la cinta bidimensional de celdas se convierte en la clase base de la cual se derivan clases para implementar las diferentes condiciones de contorno.

- `TapePeriodic`. Implementa una cinta bidimensional de tamaño fijo con condiciones de frontera periódica. Esto es, cuando una hormiga sale por uno de los bordes reaparece por el borde opuesto.
 - `TapeReflective`. Implementa una cinta bidimensional de tamaño fijo con condiciones de frontera reflectiva. Esto es, cuando una hormiga sale por uno de los bordes reaparece por el mismo borde con la orientación opuesta.
 - `TapeSliding`. Implementa una cinta bidimensional de tamaño dinámico. Esto es, cuando una hormiga sale por uno de los bordes se incrementa el tamaño de la cinta por dicho borde. Para simplificar la implementación de este tipo de cinta se define una tipo genérico de datos que almacena elementos del mismo tipo y tiene un tamaño que puede variar dinámicamente.
- `SlidingVector<T>`. Se define este tipo genérico, plantilla de datos, para representar una estructura de datos unidimensional que almacena los elementos del tipo `T` indexados con un rango de valores enteros, tanto positivos como negativos. De esta forma, la posición que ocupa un elemento no se ve afectada al añadir nuevos elementos por el principio de la estructura, puesto que el nuevo elemento se indexa con un valor decreciente del índice.

El comportamiento dinámico requerido en este tipo genérico de datos se corresponde con el comportamiento implementado en la plantilla `std::vector<T>` de la Standard Template Library (STL) [5].

- Añadir nuevo elemento por el principio: `vector<T>::insert()`
- Añadir nuevo elemento por el final: `vector<T>::push_back()`

Por tanto, un objeto `SlidingVector<T>` encapsula a un objeto `std::vector<T>` mapeando el valor del índice desde el rango `[IndexMin..IndexMax]` hasta un valor de índice en el rango `[0..IndexMax-IndexMin]` en el `std::vector<T>` encapsulado.

Por ejemplo, un objeto `SlidingVector<T> sv(-2,3)` con índices en el rango `[-2..3]` encapsula un objeto `std::vector<T> v(6)` con índices en el rango `[0..5]`

```
sv[-2] se almacena en v[0]
sv[-1] se almacena en v[1]
sv[0] se almacena en v[2]
sv[1] se almacena en v[3]
sv[2] se almacena en v[4]
sv[3] se almacena en v[5]
```

La clase `SlidingVector<T>` sobrecarga el operador de indexación, `operator[]`, para acceder a los elementos que contiene. Un intento de acceder a una posición fuera del rango `[IndexMin..IndexMax]` se considera una situación excepcional que se señala mediante una excepción del tipo `SVException`. Esta excepción será capturada por el objeto cinta, que será el responsable de aumentar el tamaño de la cinta en el borde donde se ha alcanzado la condición de frontera.

- `Ant`, es la clase base en la jerarquía de las hormigas. Se le añade un nuevo atributo, `LifeTime`, para representar un contador del número de pasos que puede realizar la

hormiga. Es por tanto un atributo que mide el tiempo de vida de la hormiga. En el constructor de cada tipo de hormiga se inicializa este atributo con un valor determinado para ese tipo, y en cada paso de la simulación se decrementa el contador hasta alcanzar el valor 0, que representa la muerte de la hormiga. En cada tipo de hormiga se definen reglas que permiten incrementar el contador del tiempo de vida.

Se amplía la jerarquía de las hormigas añadiendo un nivel intermedio donde se distinguen:

- Hormigas herbívoras. Este tipo de hormigas se caracteriza por incrementar su tiempo de vida con el valor numérico asignado a cada color de celda por la que transita. A partir del tipo hormiga herbívora se derivan distintos tipos de hormigas cuyas reglas de movimiento se corresponden con las implementadas en la práctica 2 y nombradas con la nomenclatura propuesta de Gale [3] con el prefijo "H-".
- Hormigas carnívoras. En este tipo de hormigas el incremento del tiempo de vida no depende de las celdas por las que transita, sino del tiempo de vida de las otras hormigas con las que coincide en una misma celda al finalizar un paso. El tipo de hormiga carnívora dispone de otro atributo que indica su voracidad, `Voracity`. Esto es, el porcentaje del tiempo de vida que quita a otra hormiga con la que coincide en una celda. A partir del tipo hormiga carnívora se derivan distintos tipos de hormigas con distinta voracidad, y cuyo movimiento se define utilizando las mismas reglas de giro de la propuesta de Gale [3], pero a diferencia de las hormigas herbívoras, el movimiento se realiza en diagonal. Se nombran con el prefijo "C-".
- El objeto `Simulator` sigue siendo el responsable de inicializar y evolucionar la simulación. En el fichero con los datos de inicialización se realizan los siguientes cambios:

Línea 1: Tamaño de la cinta y número de colores de las celdas

Línea 2: Tipo, posición inicial y orientación de cada hormiga (separadas por `;`)

Línea 3...: Posición y color de las celdas no blancas

Por ejemplo:

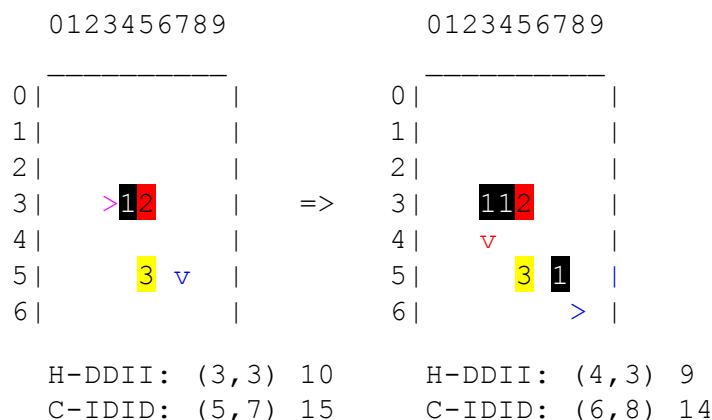
Línea 1: 7 10 4

Línea 2: H-DDII 3 3 > ; C-IDID 5 7 v

Línea 3: 3 4 1

Línea 4: 3 5 2

Línea 5: 5 5 3



Además de mostrar el número del paso de simulación, se mostrará la información de cada hormiga: tipo, posición y tiempo de vida.

El objeto simulador también asume la responsabilidad de aplicar las reglas de interacción entre las hormigas. En cada paso de la simulación, una vez que todas las hormigas han realizado su movimiento, el simulador identifica las celdas las hormigas que coinciden en una misma celda y aplica la reglas para actualizar el tiempo de vida, y si fuese necesario, para eliminar a las hormigas que hayan muerto.

Se cambia la condición de parada de la simulación, que se detiene cuando no queden hormigas vivas o cuando el usuario decida detener la simulación.

- El programa principal recibe por línea de comandos el nombre de un fichero con los datos de inicialización, que pasa al constructor del simulador para que construya e inicialice la cinta y las hormigas. Antes de terminar el programa se pregunta al usuario si desea salvar el estado de la simulación en un fichero de texto con el formato dado.

Durante las sesiones de laboratorio se podrán proponer modificaciones y mejoras en el enunciado de la práctica.

5. Referencias

[1] Práctica 1. AyEDA curso 2025-2026:

<https://campusvirtual.ull.es/2526/ingenieriaytecnologia/mod/resource/view.php?id=28424>

[2] Práctica 2. AyEDA curso 2025-2026:

<https://campusvirtual.ull.es/2526/ingenieriaytecnologia/mod/resource/view.php?id=28428>

[3] Gale, D.; Propp, J.; Sutherland, S.; Troubetzkoy, S. (1995). «Further Travels with my Ant». Mathematical Intelligencer 17: 48-56.

[4] Condición de carrera [Wikipedia]: https://es.wikipedia.org/wiki/Condici%C3%B3n_de_carrera

[5] Referencia a `std::vector` [cplusplus]: <https://cplusplus.com/reference/vector/vector/>